CODING · AULA 13 · 10 DE NOVEMBRO DE 2026

A vitrine conversa com a API

Assincronismo e integração: esperar sem travar a página, buscar e enviar dados com fetch, e o CORS. Hoje a tela e o servidor que vocês construíram passam a ser uma aplicação só.

Faculdade Senac Pernambuco · ADS 2026.2 · Prof. Sebastião Rogério

ONDE ESTAMOS NO SEMESTRE · 1 DE 3

O caminho até aqui

U1 · ✓ CONCLUÍDA

Fundamentos

11/08
Por que tantas linguagens

18/08
Paradigmas

25/08
Tipos e coleções

01/09
Funções, módulos e testes

U2 · ✓ CONCLUÍDA

Orientação a objetos

08/09
Classes

22/09
AV1 e encapsulamento

29/09
Herança e polimorfismo

06/10
Contratos e a Loja de Roupas

U3 · ✓ CONCLUÍDA

Back-end e API

13/10
HTTP, REST e a primeira API

20/10
Persistência e camadas

U4 · ◂ ESTAMOS AQUI

Front-end

27/10
Entrega 1, HTML e CSS

03/11
JavaScript, DOM e eventos

10/11
Hoje: assincronismo e integração

U5

Entrega e avaliação

17/11
PROJETO FINAL

24/11
AV2

01/12
Framework (sem nota)

08/12
Encerramento

ONDE ESTAMOS NO SEMESTRE · 2 DE 3

A última ligação que faltava

Telas no navegador	HTML e CSS na Aula 11, comportamento na Aula 12	✓ pronto
▲ fetch: a resposta sobe · o pedido desce ▼ ◀ HOJE		
API: HTTP + REST	a porta da loja para a rede · Aula 9	✓ pronto
▲ chama ▼		
Serviço e repositório	as camadas entre a porta e o banco · Aula 10	✓ pronto
▲ usa ▼		
Domínio e banco	as classes e o loja.db · Aulas 5 a 10	✓ pronto

- Todas as camadas existem e funcionam **separadas**: a API responde no `/docs`, a vitrine responde ao clique.
- Mas a vitrine ainda mostra uma lista **escrita dentro do app.js**, e o banco nem sabe que ela existe.
- Hoje entra a seta do meio: a tela **pede** os dados à API e **envia** o que a pessoa cadastrou. É o que o diagrama de arquitetura da documentação do projeto final desenha.

ONDE ESTAMOS NO SEMESTRE · 3 DE 3

Uma semana para o Projeto Final

DATA	AULA	TEMA	O QUE ACRESCENTA AO PROJETO
03/11	12	JavaScript, DOM e eventos	as telas reagem ao usuário
10/11	**13**	**Assincronismo e integração**	**hoje: as telas conversam com a API**
17/11	14	**PROJETO FINAL**	pitch de 8 minutos com demonstração ao vivo e a documentação: 50% do segundo processo
24/11	15	**AV2** e devolutiva	prova: 50% do segundo processo
01/12 e 08/12	16 e 17	Framework e encerramento	conteúdo complementar, sem nota

- A demonstração ao vivo do dia 17 é exatamente o que vocês terminam hoje: cadastrar pela tela e o produto continuar lá depois de recarregar.

O QUE VAMOS VER HOJE

Da tela ao banco, e de volta

ABERTURA

De onde viemos

retomada da Aula 12

- O produto que some
- Por que esperar é diferente

MUNDO 1

Esperar

teoria com exemplos

- Esperar sem travar
- Promessas
- async e await

MUNDO 2

Falar com a API

teoria com exemplos

- Buscar com fetch
- Enviar com POST
- CORS

MUNDO 3

Prática na loja

vocês seguem a aula

- A vitrine da API
- O cadastro grava
- Carregando e erro

ABERTURA · RETOMADA DA AULA 12

Cadastrem um produto e recarreguem a página. Cadê ele?

front/app.js · Aula 12

```
const produtos = [
  { nome: "Camiseta básica", preco: 39.9, tam: "M" },
  { nome: "Calça jeans",     preco: 129.9, tam: "G" }
];

// no submit do formulário:
produtos.push({ nome, preco, tam: "M" });
desenhar(produtos);
```

- A lista mora **na memória do navegador**. Recarregar a página joga tudo fora e começa de novo do arquivo.
- Enquanto isso, a API de vocês tem um **banco de verdade**: o `loja.db`, com `GET` e `POST /produtos` prontos desde a Aula 10.
- Falta a tela **pedir** e **enviar** os dados para essa API. Só que pedir algo pela rede leva tempo, e o navegador não pode ficar parado esperando.
- Por isso a aula começa por **como o JavaScript espera**, antes de falar de rede.



O MAPA DE HOJE

Nove fases, três mundos

Mundos 1 e 2: como o JavaScript espera e como ele conversa com a API, com exemplos. Mundo 3: a vitrine da Loja de Roupas ligada ao banco, fase por fase, com checkpoints.



FASE 1-1 · CONTEÚDO NOVO

Esperar sem travar

Por que o navegador não pode ficar parado esperando a resposta da rede.

CONCEITO AULA 13

Síncrono e assíncrono

O código que vocês escreveram até aqui é **síncrono**: uma linha só começa quando a anterior termina. Isso funciona para contas, que levam microssegundos. Um pedido pela rede leva centenas de milissegundos, às vezes segundos. Código **assíncrono** é o que dispara a tarefa demorada, **segue em frente** e trata o resultado quando ele chegar.

exemplo

```
síncrono                    assíncrono
--------                    ----------
pede o café                 pede o café e pega a senha
fica no balcão esperando    senta e conversa
recebe e só então senta     chamam a senha: busca o café

a fila trava                o salão continua andando
```

PENSE ASSIM

O garçom que fica parado na cozinha até o prato sair atende uma mesa por vez. O bom garçom anota, entrega na cozinha e volta ao salão; quando o prato fica pronto, ele leva.

CONCEITO AULA 13

Uma linha de execução só

O JavaScript da página roda numa **linha de execução só** (em inglês, *single thread*): a mesma que desenha a tela e responde ao clique. Se o código ficasse parado esperando a rede, **a página inteira congelaria**: nada de rolar, digitar ou clicar. Por isso tudo o que demora no navegador é assíncrono: o pedido fica com o navegador, e o seu código é avisado depois.

exemplo

```
console.log("1. pedi os produtos");

setTimeout(() ⇒ {
  console.log("3. chegaram");      // roda depois
}, 1000);

console.log("2. sigo funcionando");

// no console: 1, 2 e, um segundo depois, 3
```

CUIDADO

A ordem em que as linhas estão escritas **não é** a ordem em que elas rodam. É a primeira coisa que confunde quem vem do Python síncrono, e a causa de metade dos bugs desta aula.

O que o console mostra, e em que ordem?

- `console.log("A");`
- `setTimeout(() ⇒ console.log("B"), 0);`
- `console.log("C");`
- Repare no zero: o `setTimeout` pede para esperar **0 milissegundos**.

A, C, B.

Mesmo com zero, a função do `setTimeout` vai para a fila e só roda quando o código atual termina.

Assíncrono não quer dizer "demorado": quer dizer "**depois**, quando a linha de execução estiver livre".



FASE 1-2 · CONTEÚDO NOVO

Promessas

O objeto que representa um resultado que ainda não chegou.

CONCEITO AULA 13

Promise: a senha do pedido

Uma *Promise* (promessa) é um objeto que representa um valor que **vai chegar depois**. Ela tem três estados: **pendente** (*pending*, ainda esperando), **cumprida** (*fulfilled*, chegou o valor) e **rejeitada** (*rejected*, deu erro). Toda função do navegador que conversa com a rede devolve uma promessa na hora, e o valor de verdade depois.

console do navegador

```
> const p = fetch("http://127.0.0.1:8000/produtos")
> p
Promise {<pending>}        // a senha, não o café

// um instante depois:
> p
Promise {<fulfilled>: Response}

// com a API desligada:
Promise {<rejected>: TypeError: Failed to fetch}
```

PENSE ASSIM

A promessa é a senha do restaurante: você pega na hora, mas não dá para comer a senha. O que interessa é o que ela vira quando chamam o seu número.

CONCEITO AULA 13

then e catch: o jeito antigo de esperar

Antes do `await`, esperava-se uma promessa encadeando funções: `then` (então) recebe o valor quando ela é cumprida, `catch` (pegar) recebe o erro quando ela é rejeitada. Vocês vão encontrar esse estilo em tutoriais e em código antigo, então vale reconhecer, mesmo que hoje a gente escreva de outro jeito.

exemplo

```
fetch("http://127.0.0.1:8000/produtos")
  .then(resposta ⇒ resposta.json())
  .then(produtos ⇒ console.log(produtos))
  .catch(erro ⇒ console.log("falhou:", erro));

console.log("isto aparece primeiro");
```

CUIDADO

Cada `then` recebe **o resultado do anterior**. Com três ou quatro passos a corrente vira uma escada difícil de ler, e foi isso que levou a linguagem a criar o `async` e o `await`.

Em que estado fica a promessa do `fetch` em cada caso?

- a) Logo depois de chamar o `fetch`, na linha seguinte.
- b) A API respondeu com a lista de produtos.
- c) A API está desligada e o navegador não conseguiu conectar.
- d) A API respondeu **404**, produto não encontrado.

a) pendente. A rede ainda não respondeu.

b) cumprida, com a resposta dentro.

c) rejeitada, com `TypeError: Failed to fetch`.

d) cumprida! Para o `fetch`, a rede funcionou: houve resposta. O 404 é um assunto **seu**, e a fase 2-1 mostra como tratar.



FASE 1-3 · CONTEÚDO NOVO

async e await

Escrever código que espera como se fosse síncrono, sem travar a página.

CONCEITO AULA 13

async e await

`await` (aguardar) pausa **só a função** até a promessa se resolver e entrega o valor de dentro dela. A página continua livre enquanto isso. A condição: `await` só vale dentro de uma função marcada com `async`. O resultado é um código que se lê de cima para baixo, como o Python de vocês.

front/app.js

```
async function carregar() {
  const resposta = await fetch(`${API}/produtos`);
  const produtos = await resposta.json();
  console.log(produtos);           // a lista de verdade
}

carregar();
console.log("a página segue viva");
```

PENSE ASSIM

O `await` é o garçom esperando o prato **daquela mesa**. As outras mesas, isto é, os cliques, a digitação e a rolagem, continuam sendo atendidas.

CONCEITO AULA 13

O await esquecido

Sem o `await`, a variável recebe **a promessa**, e não o valor. Nada quebra na hora: o erro só aparece duas linhas depois, quando o código tenta usar uma lista que na verdade é uma senha. E toda função `async` também devolve uma promessa, então quem chama precisa de `await` se quiser o resultado.

exemplo

```
const resposta = fetch(`${API}/produtos`);   // faltou await
console.log(resposta);
// Promise {<pending>}

const produtos = resposta.json();
// TypeError: resposta.json is not a function

// certo:
const resposta = await fetch(`${API}/produtos`);
```

CUIDADO

Viu `Promise {<pending>}` no console, ou `is not a function` logo depois de um `fetch`? Procure o `await` que faltou.

CONCEITO AULA 13

Erros com try e catch

Dentro de uma função `async`, uma promessa rejeitada vira um **erro comum**, que se trata com `try` e `catch`, como o `try` e `except` do Python. É assim que a página mostra uma mensagem amigável em vez de ficar em branco quando a API está fora do ar.

exemplo

```python
# Python
try:
    r = requests.get(url)
except Exception as erro:
    print("falhou", erro)
```

```javascript
// JavaScript
try {
  const r = await fetch(url);
} catch (erro) {
  console.log("falhou", erro);
}
```

CUIDADO

O `catch` só pega o que foi **aguardado** dentro do `try`. Um `fetch` sem `await` escapa do `try` e o erro aparece solto, em vermelho, no console.

Leia o código e diga o que cada `console.log` imprime.

- `async function total() { return 42; }`
- a) `console.log(total());`
- b) `console.log(await total());`, dentro de outra função `async`
- c) `const r = fetch(url);`
  `console.log(r.ok);`

a) `Promise {<fulfilled>: 42}`. Função `async` sempre devolve uma promessa, mesmo com `return 42`.

b) `42`. O `await` abre a promessa e entrega o valor.

c) `undefined`. Sem `await`, `r` é a promessa, e promessa não tem `ok`. É o bug silencioso da fase.



FASE 2-1 · CONTEÚDO NOVO

Buscar com fetch

O pedido GET saindo do navegador e a resposta chegando como objeto.

CONCEITO AULA 13

fetch: o GET do navegador

`fetch` (buscar) faz um pedido HTTP e devolve uma promessa de *Response* (resposta). São **dois** `await`: o primeiro espera os cabeçalhos chegarem; o segundo, `resposta.json()`, espera o corpo e converte o JSON em array e objetos de JavaScript. É o mesmo JSON que vocês viam no `/docs` da API.

front/app.js

```
const API = "http://127.0.0.1:8000";

async function carregar() {
  const resposta = await fetch(`${API}/produtos`);
  console.log(resposta.status);      // 200
  const produtos = await resposta.json();
  console.log(produtos[0].nome);     // "Camiseta básica"
}

// com filtro, a query string da Aula 9:
await fetch(`${API}/produtos?tamanho=M`);
```

PENSE ASSIM

É o mesmo pedido que o `TestClient` fazia nos testes da Aula 9: `cliente.get("/produtos")`. Só que agora quem pede é a página, no navegador da pessoa.

FASE 2-1 · PASSO A PASSO

A viagem de um GET, da tela ao banco e de volta

#	O QUE ACONTECE	ONDE
1	A página carrega e chama `carregar()`	navegador · `app.js`
2	`fetch` envia `GET /produtos` e o código fica no `await`	rede
3	A rota do FastAPI chama o serviço, que chama o repositório	servidor · `api.py`
4	O repositório lê a tabela `produto`	banco · `loja.db`
5	A resposta volta como JSON, com status 200	rede
6	**O `await` termina, `resposta.json()` vira array e `desenhar` monta os cartões**	**navegador · vocês**

- Esta tabela é o diagrama de arquitetura em camadas da documentação do projeto final, escrito em linhas. Quem consegue explicá-la sabe defender o próprio diagrama.

CONCEITO AULA 13

404 não é erro para o fetch

O `fetch` só rejeita a promessa quando **não houve resposta**: API desligada, endereço errado, sem rede. Se a API respondeu, mesmo com 404 ou 500, a promessa é cumprida. Quem decide se deu certo é você, olhando `resposta.ok`, que vale `true` para os status de 200 a 299.

front/app.js

```
const resposta = await fetch(`${API}/produtos/99`);

resposta.status    // 404
resposta.ok        // false

if (!resposta.ok) {
  throw new Error(`a API respondeu ${resposta.status}`);
}
const produto = await resposta.json();
```

CUIDADO

Sem checar `resposta.ok`, o código segue feliz e tenta desenhar `{"detail": "produto 99 não encontrado"}` como se fosse um produto.



FASE 2-2 · CONTEÚDO NOVO

Enviar com POST

Mandar o que a pessoa digitou para a API gravar no banco.

CONCEITO AULA 13

POST com JSON

Para enviar, o `fetch` recebe um segundo argumento com três coisas: o `method` (método), os `headers` (cabeçalhos) e o `body` (corpo). O cabeçalho `Content-Type` avisa que vai JSON, e `JSON.stringify` converte o objeto em texto. Os campos são **os mesmos** do `ProdutoEntrada` da API.

front/cadastro.js

```
const dados = { tipo: "Produto", nome: "Boné",
                preco: 49.9, tamanho: "G" };

const resposta = await fetch(`${API}/produtos`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(dados),
});

resposta.status            // 201, Created
const novo = await resposta.json();
novo.id                    // o número que o banco deu
```

PENSE ASSIM

`JSON.stringify` é o `json.dumps` do Python; `resposta.json()` é o `json.loads`. Na rede só passa texto.

CONCEITO AULA 13

Ler o erro que a API devolve

Quando a API recusa, ela explica no corpo da resposta, no campo `detail`: foi assim que vocês escreveram os `exception_handler` da Aula 10. O front-end só precisa ler esse campo e mostrar na tela. A regra continua **no servidor**; a tela apenas avisa.

front/cadastro.js

```
const corpo = await resposta.json();

if (!resposta.ok) {
  aviso.textContent = `Não cadastrou: ${corpo.detail}`;
  return;
}
aviso.textContent = `${corpo.nome} cadastrado com o número ${corpo.id}.`;

// 409: Não cadastrou: já existe um produto chamado Boné
// 422: quando os dados não batem com o ProdutoEntrada
```

CUIDADO

Validar no front-end, como na Aula 12, é conforto para quem usa; validar no servidor é **segurança**. Qualquer pessoa pode chamar a API sem passar pela sua tela.

Os quatro POSTs abaixo falham. O que está errado em cada um?

- a) `body: dados`, sem `JSON.stringify`
- b) sem a linha do `headers`
- c) `preco: document.querySelector("#preco").value`
- d) `{ nome: "Boné", valor: 49.9, tamanho: "G" }`

a) O navegador manda `[object Object]` como texto. A API não consegue ler o JSON: **422**.

b) Sem `Content-Type: application/json`, o FastAPI não trata o corpo como JSON: **422**.

c) Funciona por sorte: `.value` é texto e o Pydantic converte `"49.9"`. Mas mande `Number(...)`: o contrato é número.

d) O campo se chama `preco` no `ProdutoEntrada`, não `valor`. Falta o obrigatório: **422**.



FASE 2-3 · CONTEÚDO NOVO

CORS

Por que o navegador bloqueia a própria API de vocês, e quem libera.

CONCEITO AULA 13

Origem: protocolo, endereço e porta

Para o navegador, uma **origem** é a soma de protocolo, endereço e porta. Por segurança, uma página só lê respostas da **mesma origem** em que foi aberta. Na aula, as páginas rodam na porta **5500** e a API na **8000**: para o navegador, são dois sites diferentes.

exemplo

```
página:  http://127.0.0.1:5500/index.html
API:     http://127.0.0.1:8000/produtos
                        ^^^^ porta diferente = outra origem

// no console, em vermelho:
Access to fetch at 'http://127.0.0.1:8000/produtos'
from origin 'http://127.0.0.1:5500' has been blocked
by CORS policy: No 'Access-Control-Allow-Origin'
header is present on the requested resource.
```

PENSE ASSIM

A regra protege o usuário: sem ela, qualquer site aberto numa aba poderia ler dados de outro site em que a pessoa está logada.

CONCEITO AULA 13

CORS: quem libera é o servidor

CORS é *Cross-Origin Resource Sharing*, compartilhamento de recursos entre origens. É a forma de a API dizer ao navegador quais origens podem ler as respostas dela. A liberação vai **no servidor**, com um cabeçalho na resposta; mudar o `fetch` não resolve. No FastAPI, são algumas linhas no `api.py`.

api.py

```
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI(title="Loja de Roupas", lifespan=ciclo_de_vida)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:5500",
                   "http://localhost:5500"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

CUIDADO

`allow_origins=["*"]` faz o erro sumir, e libera a API para qualquer site da internet. Liste as origens de vocês. E `127.0.0.1` e `localhost` são origens **diferentes**: liste as duas.

FASE 2-3 · BLOQUEIA OU NÃO?

Com a API liberando só `http://127.0.0.1:5500`, a vitrine consegue ler os produtos?

- a) Página aberta em `http://127.0.0.1:5500`
- b) Página aberta em `http://localhost:5500`
- c) Página aberta com dois cliques no arquivo, `file:/// .../index.html`
- d) O mesmo pedido feito no `/docs` ou pelo `pytest`

a) lê. Origem listada.

b) bloqueia. `localhost` e `127.0.0.1` são origens diferentes para o navegador.

c) bloqueia. Arquivo aberto do disco não tem origem de rede. Rode as páginas com `python -m http.server 5500`.

d) funciona. CORS é regra **do navegador**: o `/docs` é da mesma origem da API, e o `pytest` nem é navegador.

HORA DA PRÁTICA

Agora é com vocês: dois terminais abertos

terminal 1 · a API

```
$ fastapi dev api.py
Serving at: http://127.0.0.1:8000
```

terminal 2 · as páginas

```
$ python -m http.server 5500 --directory front
Serving HTTP on 0.0.0.0 port 5500
```

- Abram a pasta `loja-roupas` no editor, a mesma da Aula 12.
- No navegador, abram `http://127.0.0.1:5500`, e não o arquivo com dois cliques.
- Os dois terminais ficam **ligados** a aula inteira. Fechou o da API, a vitrine para.
- Três fases, cada uma com checkpoint: `pytest`, commit e push.
- Deixem o console aberto (F12). Hoje também a aba **Rede** (*Network*), que mostra cada pedido e a resposta.



FASE 3-1 · PRÁTICA

A vitrine da API

Trocar a lista escrita à mão pelos produtos do banco, e liberar o CORS.

FASE 3-1 · NO EDITOR

front/app.js: a lista sai, o fetch entra

front/app.js

```
const API = "http://127.0.0.1:8000";

let produtos = [];          // agora vem da API

function montarCard(p) {
  const art = document.createElement("article");
  art.innerHTML = `<h3>${p.nome}</h3>
                   <p>Tamanho ${p.tamanho}</p>
                   <p><strong>R$ ${p.preco.toFixed(2)}</strong></p>`;
  return art;
}

async function carregar() {
  const resposta = await fetch(`${API}/produtos`);
  produtos = await resposta.json();
  desenhar(produtos);
}

carregar();     // no lugar de desenhar(produtos)
```

- A lista escrita à mão **sai**. `let`, porque agora ela é preenchida depois.
- A API chama o campo de `tamanho`, e não `tam`. **O front segue o contrato da API**, nunca o contrário.
- `desenhar` e a busca da Aula 12 **não mudam**: a busca filtra a lista que veio do banco.
- Salvem e recarreguem: o console vai mostrar o erro de CORS. É esperado, e é o próximo slide.

FASE 3-1 · NO EDITOR

api.py: liberar a origem das páginas

api.py

```
from fastapi import Depends, FastAPI
from fastapi.middleware.cors import CORSMiddleware
...
app = FastAPI(title="Loja de Roupas", lifespan=ciclo_de_vida)

# as páginas rodam na porta 5500; o servidor diz quem pode chamar
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:5500", "http://localhost:5500"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- O `fastapi dev` recarrega sozinho quando o `api.py` é salvo.
- Recarreguem a vitrine: os produtos agora vêm do `loja.db`.
- Na aba Rede do F12, cliquem no pedido `produtos`: o cabeçalho `access-control-allow-origin` aparece na resposta.
- O teste `tests/test_cors.py`, no gabarito, prova as duas coisas: a porta 5500 ganha permissão e um site estranho não.

FASE 3-1 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
tudo verde
$ git add .
$ git commit -m "Fase 3-1: a vitrine busca os produtos na API"
$ git push
```

CONFIRA

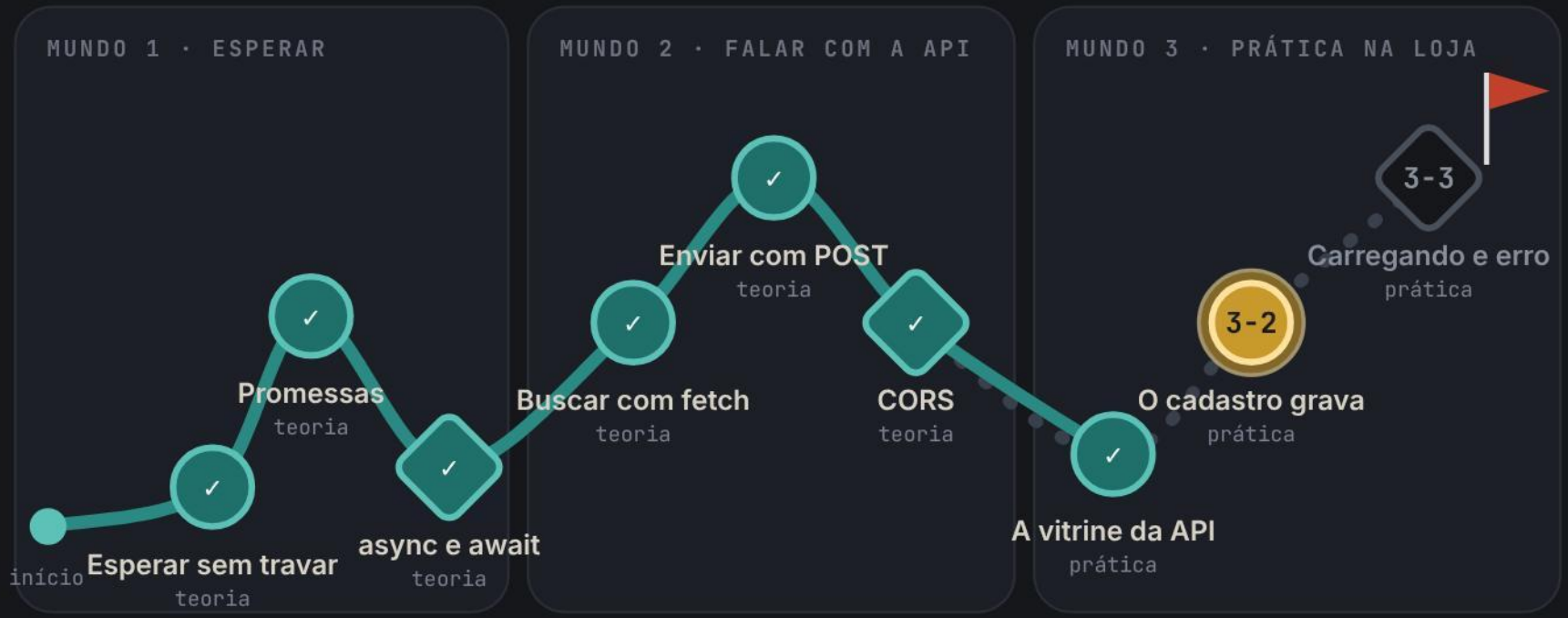
A vitrine mostra os produtos do banco. Prova: cadastrem um produto pelo `/docs`, recarreguem a vitrine, e ele aparece. Console sem vermelho.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-22.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-22`



FASE 3-2 · PRÁTICA

O cadastro grava

O formulário da Aula 11 envia para a API, e o produto passa a existir de verdade.

FASE 3-2 · NO EDITOR

front/cadastro.js: do formulário ao banco

front/cadastro.js

```
const API = "http://127.0.0.1:8000";
const form = document.querySelector("#form-produto");
const aviso = document.querySelector("#aviso");
const campo = (id) => document.querySelector(id).value;

form.addEventListener("submit", async (e) => {
  e.preventDefault();
  const dados = { tipo: campo("#tipo"), nome: campo("#nome").trim(),
                  preco: Number(campo("#preco")), tamanho: campo("#tamanho") };
  const resposta = await fetch(`${API}/produtos`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dados),
  });
  const corpo = await resposta.json();
  if (!resposta.ok) {
    aviso.textContent = `Não cadastrou: ${corpo.detail}`;
    return;
  }
  aviso.textContent = `${corpo.nome} cadastrado com o número ${corpo.id}.`;
  form.reset();
});
```

- A função do `submit` ganhou `async`: sem ele, o `await` lá dentro é erro de sintaxe.
- No `cadastro.html`: `id="form-produto"` no `form`, um `<p id="aviso" role="status">` depois dele e o `<script src="cadastro.js">` no fim do `body`.
- Os quatro campos são os do `ProdutoEntrada`, e o teste da Aula 11 já vigiava isso.

FASE 3-2 · NO NAVEGADOR

Conferir como um profissional

#	O QUE FAZER	O QUE TEM QUE ACONTECER
1	Cadastrar **Boné**, R$ 49,90, tamanho G	aviso com o número que o banco deu, e o formulário limpo
2	Voltar à vitrine e recarregar com F5	o Boné está lá, e continua depois de recarregar
3	Cadastrar Boné de novo	"Não cadastrou: já existe um produto chamado Boné", o 409 da Aula 10
4	Parar a API com Ctrl+C e ligar de novo	o Boné continua: está no `loja.db`
5	Aba Rede do F12, pedido `produtos` do tipo POST	status 201 e o JSON do produto novo na resposta

- O passo 4 é a demonstração do Projeto Final: o dado sobrevive ao servidor desligar.

FASE 3-2 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
tudo verde
$ git add .
$ git commit -m "Fase 3-2: o cadastro grava no banco pelo POST"
$ git push
```

CONFIRA

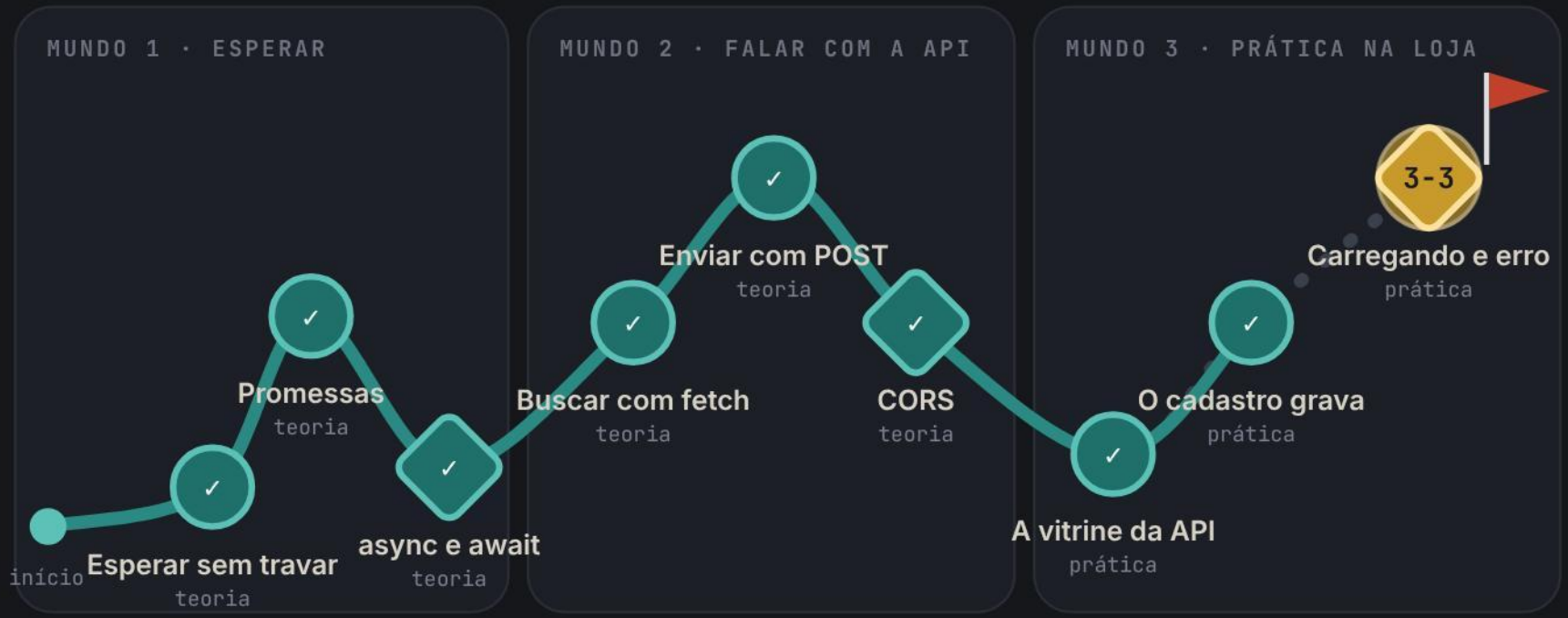
Os cinco passos da conferência funcionam: o produto cadastrado aparece na vitrine, sobrevive ao F5 e ao servidor reiniciar, e a repetição mostra o 409 na tela.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-23.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-23`



FASE 3-3 · PRÁTICA

Carregando e erro

A tela diz o que está acontecendo, em vez de ficar em branco.

FASE 3-3 · NO EDITOR

front/app.js: carregando, vazio e API fora do ar

front/app.js

```
// no index.html, logo abaixo do h2:
//   <p id="estado" role="status"></p>
const estado = document.querySelector("#estado");

async function carregar() {
  estado.textContent = "Carregando produtos...";
  try {
    const resposta = await fetch(`${API}/produtos`);
    if (!resposta.ok) {
      throw new Error(`a API respondeu ${resposta.status}`);
    }
    produtos = await resposta.json();
    desenhar(produtos);
    estado.textContent = produtos.length === 0
      ? "Nenhum produto cadastrado ainda." : "";
  } catch (erro) {
    estado.textContent = "Não foi possível falar com a API. Ela está ligada?";
    console.error(erro);
  }
}
```

- Uma tela de dados tem **quatro estados**: carregando, com dados, vazia e com erro. A tela em branco não comunica nenhum.
- O `throw` manda o 404 e o 500 para o mesmo `catch` da API desligada: uma mensagem só para "não deu".
- `console.error` guarda o detalhe técnico para vocês; a pessoa lê a frase em português.
- No `cadastro.js`, o mesmo `try` e `catch` em volta do `fetch`, com a mesma mensagem no `aviso`.

FASE 3-3 · NO NAVEGADOR

Testar os quatro estados

ESTADO	COMO PROVOCAR	O QUE A TELA MOSTRA
carregando	F12 → Rede → limitação *Slow 3G* (3G lento) e F5	"Carregando produtos..." por alguns segundos
com dados	o normal, com a API ligada	os cartões, e a mensagem some
vazia	apagar os produtos pelo `DELETE` do `/docs`	"Nenhum produto cadastrado ainda."
erro	Ctrl+C no terminal da API e F5 na vitrine	"Não foi possível falar com a API. Ela está ligada?"

- Na apresentação do dia 17, a API pode cair no meio da demonstração. Com esta fase, a tela explica o que houve em vez de ficar em branco na frente da turma.

FASE 3-3 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
tudo verde
$ git add .
$ git commit -m "Fase 3-3: carregando, vazio e erro na tela"
$ git push
```

CONFIRA

Os quatro estados aparecem como na tabela. Com a API desligada, a vitrine e o cadastro mostram a mensagem em português, e não uma tela em branco.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

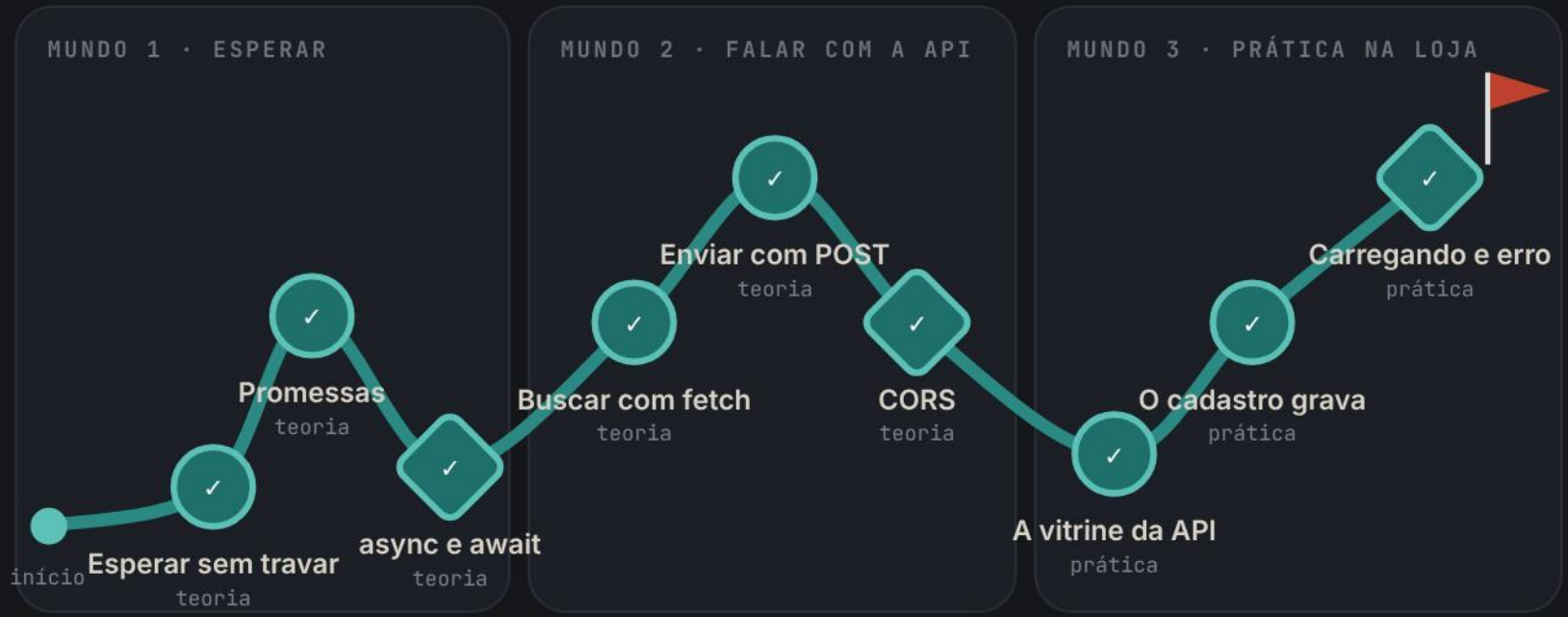
`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-24.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-24`

Mãos à obra: o projeto de ponta a ponta



- CORS liberado no `api.py` da equipe para a porta 5500.
- A listagem vinda do `GET`, com os nomes de campo da **API de vocês**.
- O cadastro pelo `POST`, mostrando o `detail` quando a API recusa.
- Os quatro estados: carregando, com dados, vazia e erro.
- Ensaio da demonstração do dia 17. Console limpo, `pytest` verde, commit e push.



O MAPA DE HOJE

Mapa completo

Nove fases fechadas: esperar sem travar, buscar, enviar e liberar o CORS, e a Loja de Roupas ligada de ponta a ponta, da tela ao banco.

FECHAMENTO

O que cada fase construiu

FASE	CONCEITO	O QUE PROVA
1-1	síncrono, assíncrono e a linha de execução única	em que ordem aparece
1-2	*Promise*: pendente, cumprida, rejeitada; `then` e `catch`	o estado de cada promessa
1-3	`async`, `await`, `try` e `catch`	o que o console imprime
2-1	`fetch`, `resposta.json()`, `resposta.ok`	a viagem de um GET em seis passos
2-2	POST com `method`, `headers` e `JSON.stringify`	por que deu 422
2-3	origem e CORS, liberados no servidor	bloqueia ou não bloqueia
3-1	a vitrine vinda do banco	o produto do `/docs` aparece na vitrine
3-2	o cadastro pelo POST	os cinco passos da conferência
3-3	os quatro estados da tela	a API cai e a tela explica

PRÓXIMA AULA · 17/11

Projeto Final: a aplicação de ponta a ponta, ao vivo

- Pitch de **8 minutos** por equipe, com **demonstração ao vivo**, seguido de revisão de código cruzada entre equipes.
- A demonstração que convence é a desta aula: cadastrar pela tela, recarregar e mostrar que o dado continua no banco.
- A **documentação** vai junto, na pasta `docs/` do repositório: diagrama de domínio, modelo ER, diagrama de arquitetura em camadas e relatório técnico. Push até as 18h do dia 17.
- **Antes de sair hoje:** os dois terminais funcionando no computador de quem vai apresentar.
- Ensaiem com a API desligada: é o tipo de imprevisto que acontece no dia.
- O diagrama de arquitetura em camadas é a tabela "a viagem de um GET" desenhada. Usem a da fase 2-1 como roteiro.

PARA ESTUDAR

Referências

- **Trilha do Navegador · Parte B**: DOM, eventos e formulários, no site da disciplina, para rever a base da Aula 12 antes de ligar na API.
- **MDN Web Docs**, em português: as páginas "Usando Fetch", "async function" e "CORS". Endereço: `developer.mozilla.org/pt-BR`
- **javascript.info**: os capítulos "Promises, async/await" e "Network requests".
- **Documentação do FastAPI**: a página "CORS (Cross-Origin Resource Sharing)", com o mesmo `CORSMiddleware` da aula.
- O **gabarito** da Loja de Roupas tem as três fases de hoje nas etapas 22, 23 e 24.
- Travou na API da equipe? Comparem com o gabarito arquivo por arquivo: `app.js`, `cadastro.js` e o trecho do CORS no `api.py`.

PARA DEIXAR ABERTO

Cola da aula

QUERO…	ESCREVO
uma função que espera	`async function carregar() { ... }`
esperar uma promessa	`const x = await algo();`
buscar na API	`const r = await fetch(`${API}/produtos`);`
ler o JSON da resposta	`const dados = await r.json();`
saber se deu certo	`r.ok` e `r.status`
enviar um POST	`fetch(url, { method: "POST", headers: {...}, body: ... })`
avisar que vai JSON	`headers: { "Content-Type": "application/json" }`
objeto vira texto	`JSON.stringify(dados)`
tratar a API fora do ar	`try { ... } catch (erro) { ... }`
mostrar o erro da API	`aviso.textContent = corpo.detail`
liberar o CORS	`app.add_middleware(CORSMiddleware, allow_origins=[...])`
servir as páginas	`python -m http.server 5500 --directory front`